



Learning evolving prototypes for imbalanced data stream classification with limited labels

Zhonglin Wu^a, Hongliang Wang^a, Jingxia Guo^a, Qinli Yang^a, Junming Shao^{b,c,a,*}

^a Data Mining Lab, School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China

^b Yangtze Delta Region Institute (Quzhou), University of Electronic Science and Technology of China, Huzhou, China

^c Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Shenzhen, China

ARTICLE INFO

Keywords:

Data streams

Concept drift

Imbalanced learning

Active learning

ABSTRACT

Real-world data streams often exhibit long-tailed distributions with heavy class imbalance, posing great challenges for data stream classification, especially in the case of label scarcity and concept drift. Several active learning methods have been proposed to address this problem by selecting the most valuable instances for labeling. However, existing methods often struggle to dynamically identify the most valuable instances that truly represent the current concept while still requiring a large label budget. In this work, we propose a new algorithm, LEPID, to combine dynamic micro-cluster concept modeling and local entropy modeling to select current important concepts and prototypes. Specifically, we give greater weight to concept drift prototypes and minority prototypes to focus more on those regions that represent current concepts. We use a local entropy strategy based on micro-clusters to select the most valuable instances for labeling and reduce the label budget. Extensive experiments on real-world and synthetic imbalanced datasets show that, compared to state-of-the-art algorithms, our method can naturally adapt to concept drift and dynamically capture the current and most valuable prototypes to achieve better results even in the case of label scarcity.

1. Introduction

Large-scale data streams in the real world often exhibit long-tailed label distributions, resulting in a ubiquitous class imbalance [1]. Traditional data stream algorithms without class imbalance addressing mechanisms focus on the majority class while ignoring the minority class; these algorithms can achieve high overall accuracy but perform poorly on the minority class. In addition, the evolutionary nature of data streams can lead to concept drift, i.e., the underlying data distribution may change greatly over time. Although class imbalance learning and concept drift have been extensively researched in the literature [2–4], the combined problem has received little attention. It is commonly seen in real-world applications; for example, credit card fraud detection is a difficult problem because of concept drift (i.e., customer habits shift and fraud is disguised) and class imbalance (i.e., the number of genuine

* Corresponding author at: Data Mining Lab, School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China.

E-mail addresses: zhonglin_wu@std.uestc.edu.cn (Z. Wu), hongliangwang@std.uestc.edu.cn (H. Wang), jxguo@std.uestc.edu.cn (J. Guo), qinli.yang@uestc.edu.cn (Q. Yang), junmshao@uestc.edu.cn (J. Shao).

<https://doi.org/10.1016/j.ins.2024.120979>

Received 6 March 2024; Received in revised form 27 May 2024; Accepted 5 June 2024

Available online 11 June 2024

0020-0255/© 2024 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

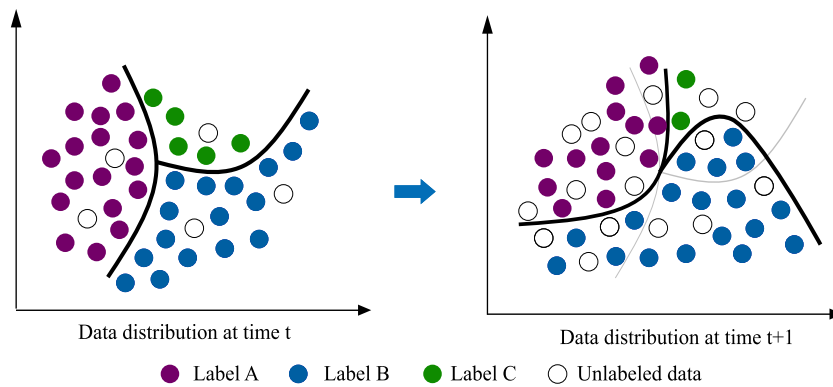


Fig. 1. The challenges of imbalanced data stream classification in the presence of concept drift and limited labels. Here we use circles of different colors to represent different classes, with green circles representing the minority class and hollow (white) circles representing unlabeled data. At time $t+1$ (right), a concept drift occurred (the decision boundary changed, the thin gray line is the previous decision boundary), and at the same time, the imbalance ratio of classes also changed.

transactions greatly outnumbers the number of frauds). Class imbalance and concept drift have mutually reinforcing effects, which means that if both issues coexist, they tend to increase each other. In other words, if the classes in the data stream are imbalanced, it will be considerably more difficult to detect the concept drifts of the minority class and adapt the online learner to them. Conversely, the occurrence of concept drift tends to alter the ratio of class imbalances. In recent years, some researchers have combined ensemble learning and resampling techniques to address this problem [5,6]. Specifically, adaptive concept drift can be achieved by changing the structure of the ensemble model and then using resampling techniques such as oversampling of the minority class to address class imbalance. However, most ensemble learning is a chunk-based approach. The primary disadvantage is that classifiers are constructed, evaluated, or updated only after all instances from a new block are accessible, which results in a delayed response to changes in some settings. In addition, ensemble learning is supervised learning that assume unrestricted access to class labels during training, which is clearly inconsistent with real-life data stream scenarios.

Furthermore, many current data stream algorithms require a large amount of labeled data to reach their full potential [7]. However, obtaining labeled data can be hard and expensive, particularly in specialized fields where trustworthy labels can only be provided by specialists whose time is valuable. For illustration, Fig. 1 provides a simple example. As shown in Fig. 1, at the initial stage, we have some initial data with three classes, one of which is minority class. A classifier is built on these initial data. However, over time, we observe that concept drift occurs, leading to changes in the decision boundary. This can affect the classification performance of the model, especially when there are limited labels. Unlabeled data near the decision boundary can make it more ambiguous, resulting in worse classification performance of the model. This is similar to the problem we explore, namely, how to learn online with limited labels when concept drifts and class imbalances occur simultaneously in the data stream. These problems are not independent but may interfere, i.e., the occurrence of concept drift can lead to changes in the state of class imbalance. Because of the scarcity of labels, choosing which data to obtain its true labels has a significant impact on the classification effect of the model. Therefore, the algorithm must use a sensible strategy to query the true labels of instances. In the method of selecting instances to be labeled, active learning has become a research hotspot in recent years because it can build predictive models using the most valuable labeled instances [8,9]. In contrast to pool-based active learning on static datasets, online active learning can timely select the most valuable instances for labeling through query strategies, update the model in real-time, improve accuracy, and better fit the data stream scenario. Although there have been some studies considering concept drift in online active learning, such as using uncertainty query strategies to label instances and then detecting concept drift using active approaches (such as ADWIN), these methods still require a relatively large label budget, typically between 20% and 60% [7]. Since a data stream is inherently unbounded, querying labels under these label budgets is still a costly task. It remains an urgent problem to conduct online active learning in the context of data streams that are labeled sparsely.

Based on the above analysis, online learning tasks would be extremely difficult for data streams that simultaneously contain class imbalances and concept drifts in the case of label scarcity because these challenges are not independent but intricately interconnected and mutually influential. To the best of our knowledge, no viable approach is currently available. In light of this, we present an active learning framework capable of tackling all three issues. The key idea is to combine different active learning strategies to dynamically model important prototypes and query the most valuable instances under the current concept. More specifically, we use a random strategy to select instances and obtain an approximate class imbalance state of the current class distribution. Then, we assign greater weight to the current concept drift micro-clusters and minority class micro-clusters, increasing the chance of selecting instances near them. We use the local neighborhood class entropy of micro-clusters to query the most valuable instances. Finally, we incorporate the true labels of the queried instances into the micro-cluster update, obtaining an efficient, flexible, and precise online model through a finer-granularity micro-cluster update approach. Major contributions are listed as follows:

- **Dynamic selection of important prototypes.** We use Concept Adaptive Strategy to dynamically assign more weight to prototypes with concept drift and minority class. In addition, we use the Local Entropy Strategy to select prototypes that are closer to the decision boundary, so that instances near these more important prototypes have a greater chance to be selected.

- **A finer granularity to update prototypes.** We adapt the method of micro-cluster splitting to update micro-clusters at a finer granularity scale. Compared to simply deleting the entire micro-cluster, this approach, while focusing on the latest concepts, retains some of the important information from the previous micro-cluster, increasing the model's sensitivity to minority classes.
- **High performance with label scarcity.** By integrating the active learning strategy with the micro-cluster update mechanism, the algorithm is able to extract the most valuable data even in situations where label availability is limited, while maintaining high predictive performance.

2. Related work

Over the last decade, numerous data stream mining algorithms have emerged [10–12]. Owing to space constraints, we have chosen to highlight only the most closely approaches found in the literature.

2.1. Imbalance data stream with concept drift

Class imbalances are a prevalent issue encountered in the domain of data stream mining [13]. When combined with concept drift, new challenges arise in the learning process. The imbalance ratio is no longer static but continuously changes along with the evolving data stream. At the same time, the occurrence of concept drift will also change the imbalance ratio [14]. Researchers have proposed many methods to address the joint problem of concept drift and class imbalance. In terms of the way instances arrive, these methods can be divided into block-based methods and online learning methods [15,16]. The chunk-based methods buffer the data stream until a specified number of samples have been acquired, after which classifiers are built on the data chunk. Usually, an ensemble framework is adopted to create a single classifier for each incoming data block and adjust the weights of the classifiers to adapt to the new concept. At the same time, the imbalance problem can be addressed by accumulating minority-class samples from past blocks. Jiao et al. [17] introduced a dynamic ensemble selection method, known as DESICD, specifically designed for imbalanced data streams with concept drift. When data arrives incrementally in chunks, they devised a novel synthetic minority oversampling technique with adaptive nearest neighbors (AnnSMOTE) to generate fresh minority instances that align with the evolving concept. Subsequently, DESICD constructs a base classifier using the newly arrived data chunk, balanced by AnnSMOTE and combines it with historical base classifiers to create a pool of candidate classifiers. For each query instance, the optimal combination is determined based on the performance of these candidate classifiers within its local neighborhood. Yang et al. [18] presented the dynamic weighted majority for imbalance learning (DWMIL) method, which adjusted the classifier weights based on the classifier's ability to distinguish between the current chunk and the creation time. Using only instances in the current chunk can help combat idea drift, but it is difficult to guarantee that sufficient minority examples exist in the current chunk. Lu et al. [19] improved the DWMIL and proposed the adaptive chunk-based dynamic weighted majority (ACDWM). The method utilizes an ensemble framework with dynamic individual classifier weights. The weights of the classifiers depend on the imbalance-aware error of the classifiers tested on the current block. Classifiers with smaller errors will receive higher weights in the ensemble. Additionally, the method can adaptively determine the appropriate block size to train individual classifiers instead of using fixed-size data blocks. All the abovementioned methods are suitable only for binary imbalance problems. Generally, block-based learning algorithms face the challenge of determining the size of data blocks. If the data block is too large, the algorithm responds slowly to sudden concept drift. If the data block is too small, the algorithm can handle sudden drift, but it will consume more computational resources. In addition, all of these are supervised learning methods, assuming unlimited access to the true labels of the data. However, due to the high cost of obtaining data labels in reality, these methods have significant limitations in real data streaming scenarios.

Unlike block-based methods, online methods process each single instance one by one, and their main advantages are short response times and strong adaptability to changes in the stream [20]. For example, Wang et al. [21] proposed Oversampling Online Bagging (OOB) and Undersampling Online Bagging (UOB) based on Online Bagging (OB). These methods can be used to classify imbalanced data streams with time-varying imbalance ratios and dynamically adjust the number of resampling instances based on real-time imbalance ratios. Wang et al. [22] introduced the multiclass oversampling online bagging (MOOB) and multiclass undersampling online bagging (MUOB) models. They analyzed it to elucidate the factors contributing to MUOB's relatively lower performance compared to MOOB. The investigation revealed that oversampling appeared to enhance performance stability, whereas undersampling, by discarding examples, appeared more likely to lead to insufficient learning in online scenarios. Ferreira et al. [23] introduced ARFR to address the complexities arising from mixed concept drift alongside varying multiclass imbalance ratios. ARFR inherited the concept drift detection mechanism from ARF and employed an approach that tracked the occurrence frequency of instances in each category right from the beginning of the data stream to assess the multiclass imbalance ratio. Subsequently, based on the current multiclass imbalance ratio, ARFR incorporated weights into the output derived from the Poisson distribution to adjust the likelihood of an instance being used for training. However, in cases of concept drift with variable imbalance ratios, especially when the minority class becomes the majority class, the computed results of the current imbalance ratio tend to become "averaged". Although DDM-OCI, OOB, and UOB can handle imbalance data streams online, these methods are designed for binary imbalance problems. MOOB and MUOB address multi-class imbalance issues, but they do not have specific mechanisms designed to handle concept drift. ARFRE considers the coexistence of multiclass imbalance and concept drift, but as the data stream evolves, its mechanism for evaluating the current multiclass imbalance ratio may become ineffective when the minority class becomes the majority class or vice versa. Additionally, both block-based and online methods are supervised. Therefore, these methods cannot be used in cases where labels are scarce.

2.2. Active learning for imbalance data stream

In recent years, active learning has garnered substantial attention within data stream mining [9,24]. In situations where labeling resources are constrained, the selection of a label request strategy plays a pivotal role in determining the efficacy of active learning and achieving cost-effective results. In imbalanced data stream, active learning can give more attention to minority classes through query strategies, thus achieving a better overall classification effect. Aguiar et al. [25] observed the connection between the occurrence of concept drift and label budget, and introduced a novel active learning strategy that dynamically manages the labeling budget by monitoring concept drift in the data stream. This adjustment gives the classifier greater flexibility to adapt to new concepts. Traditional label request strategies within the domain of active learning encompass the uncertainty strategy, random strategy, and hybrid strategy. The uncertainty strategy [24] focuses on selecting instances based on the model's assessment of predictive uncertainty for each instance. In contrast, the random strategy [26] involves randomly sampling instances from the data stream. The hybrid strategy [27], as its name suggests, combines elements from both the random and uncertainty strategies to create a comprehensive approach. In recent years, the combination of ensemble learning and active learning has emerged as a new approach to address the classification of evolving data streams. Xu et al. [28] introduced an innovative ensemble model that incorporates active learning to tackle the challenges posed by concept drift and the limited availability of labeled instances. This model consists of two fundamental classifiers: a stable classifier responsible for predicting labels for incoming instances and a reactive classifier designed to detect concept drift. In cases where concept drift is detected, the reactive classifier takes over from the stable classifier and is subsequently retrained using a random labeling strategy. Meanwhile, the stable classifier undergoes updates through a hybrid labeling strategy, which integrates elements from both the uncertainty strategy and the random strategy. Liu et al. [29] introduced an innovative uncertainty strategy that leverages an asymmetric margin threshold matrix, offering a nuanced representation of uncertainty distinctions within the multiclass imbalance context, particularly among majority and minority classes. The proposed comprehensive formula for sample weighting leans toward assigning higher training weights to recently introduced, indistinct, and minority class samples. Furthermore, it is noteworthy that recent research has explored the use of active learning in conjunction with cluster-based learning methods to address the challenges posed by concept drifting data streams. Micro-clusters were first proposed by Aggarwal et al. [30] in their Clustream algorithm, and in subsequent studies, Yin et al. [31] proposed a label query strategy that combines a marginal threshold matrix, enabling the identification and selection of challenging to classify or potentially concept drifting samples for labeling. This strategy mitigates issues related to expensive labeling costs and data imbalance. Additionally, the inclusion of a buffer is employed to store newly formed micro-clusters, facilitating their participation in model updates to accommodate evolving data environments. Different from the above methods of selecting from existing samples and online oversampling methods such as OOB, Aguiar et al. [32] proposed an active learning method based on labeling budget, which combines information from budget constraints and dynamic class ratios to generate new relevant instances. Although the above active learning methods can also achieve good classification results when only querying the true labels of a part of instances, when facing the data stream with infinite length attributes, the proportion of labels in these active learning queries is still large, and how to achieve better prediction results under more scarce labels is still an urgent problem to be solved.

Building upon the analysis of related works, it is discernible that the primary limitation of existing research resides in the absence of a comprehensive approach. This approach should encompass the interrelated complexities of multiclass imbalance, concept drift, and online active learning, all within the constraints of a limited label budget. To our best understanding, there presently exists no method that efficaciously addresses these amalgamated challenges.

3. Proposed method

Within this section, we provide a presentation of the proposed framework's overview in Section 3.1. Subsequently, we delve into the details of the initializing learning model, online active learning strategy, classification model, and model update in Sections 3.2, 3.3, 3.4, and 3.5, respectively. To conclude, we analyze the time and space complexities associated with the proposed algorithm.

3.1. Overview

The motivation of the LEPID algorithm is to dynamically capture data near areas where prone to concept drift and minority classes, which are generally more important and valuable. As introduced in Section 3.3, the basic idea of our algorithm is to combine dynamic microcluster concept modeling and local entropy modeling based on microclusters to select current important concepts and prototypes, and continuously update the model to adapt to concept drift and class imbalance in evolving data streams. Initially, the learning model, developed from the initial training data, generates K initial micro-clusters. Subsequently, during the learning phase, three active learning strategies are implemented to determine the necessity of acquiring true labels for instances. Consequently, the class distribution of these samples requested by the random strategy can be approximated as the current class imbalance status of the data stream. Only those true labels acquired during the initialization stage or via the random strategy are pushed into the label sliding window (sizelab). Next, the concept adaptive selection strategy emphasizes selecting instances of the minority class and assigning greater concept adaptation weights W_{ca} to micro-clusters of the minority class that are likely to experience concept drift. The concept adaptation weight W_{ca} follows a similar weight calculation formula as the micro-cluster weight W_{mc} , but W_{ca} has a larger λ , meaning that W_{ca} updates with greater magnitude. In subsequent selections, micro-clusters with larger W_{ca} weights, i.e., those that are of the minority class and are likely to experience concept drift, will be more likely to be selected. Finally, instances that are closer to the

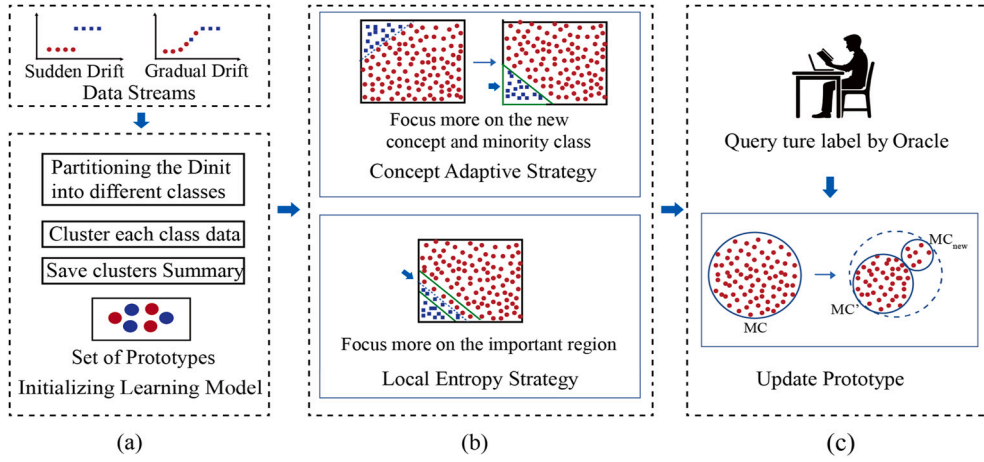


Fig. 2. The workflow of LEPID algorithm. (a) Initial stage: clustering and prototyping of the initial data (Dinit) and preserving the summary of the prototypes. (b) Query stage: dynamic querying of minority classes of instances and important instances (within the green real line) in the current concept through concept adaptive strategy and local entropy strategy. (c) Updating stage: update micro-clusters in a finer granularity, which focuses on the latest concepts while retaining some important information from previous micro-clusters.

decision boundary and difficult to distinguish are selected based on the local class entropy to dynamically adapt to changes in the decision boundary. The true labels obtained through these active learning strategies will be used to update the model. In short, LEPID uses the active learning strategy mentioned above to dynamically capture these more valuable data, give more training weight to minority micro-clusters and micro-clusters with concept drift regions, obtain the true labels of the data in these regions and add them to the model update, so that the model can better adapt to these regions. At the same time, the entire model will be continuously maintained online through mechanisms such as micro-cluster splitting and the addition and elimination of micro-clusters. Compared to ensemble learning algorithms like DESICD and ACDWM mentioned earlier, which assume data streams arrive in the form of data chunks, LEPID is an online learning algorithm that does not have the issue of setting data chunk sizes. Additionally, compared to these supervised methods, LEPID is a semi-supervised algorithm that only requires a part of labeled data to achieve good classification results. Furthermore, unlike OOB and UOB algorithms that can only be used for binary data stream classification tasks, LEPID is not only suitable for binary data stream classification tasks but also performs well in multi-class imbalanced data stream classification tasks. Finally, compared to CALMID, LEPID requires fewer labeled data (as little as 10% or 1% labeled data) to effectively complete classification tasks. Fig. 2 gives the workflow of the LEPID algorithm. In the following, we will introduce each module step by step.

3.2. Initializing learning model

LEPID defines the function that initializes the learning model in Algorithm 1. Upon the arrival of the initial training dataset, denoted as D_{init} , we employ the k-means clustering algorithm to partition it into K clusters, i.e., $C = C_1, C_2, \dots, C_K$. Instead of retaining the K clusters themselves, we calculate the statistical summarization of their clustering feature representations. Subsequently, we employ micro-clustering (MC) techniques to preserve these characteristic features. MC consists of multiple features (LS, SS, N, C_{MC} , R_{MC} , W, W_{ca} , T). LS and SS are the linear sum and square sum of each labeled class in the storage cluster, respectively, as shown in equation (1). N is a cluster feature used to store total data points in a micro-cluster. The parameter W signifies the weighting factor that governs micro-clustering during the model update process. The initial value of parameter T is initialized to zero, indicating the time elapsed since the last micro-cluster update. W_{ca} represents the weight of the concept adaptive micro-cluster. The center C_{MC} and radius R_{MC} of each micro-cluster can be obtained with equation (2) and Equation (3), respectively. By retaining and dynamically maintaining these micro-clusters, the data stream clustering algorithm can abstract the data effectively. The update of micro-clusters leverages their additive and incremental properties. Upon reaching a predetermined threshold, two micro-clusters are merged using the updated clustering features, and new instances can be incrementally added to the micro-clusters. To update the model, the phase-specific model can be modified by inserting new micro-clusters, removing obsolete ones, or dynamically updating existing micro-clusters.

$$LS^l = \sum_{j=1}^{|C_i^l|} x_j, \quad SS^l = \sum_{j=1}^{|C_i^l|} x_j^2, \quad (C_i^l \in C_i) \quad (1)$$

$$C_{MC} = \frac{\sum_{l=1}^L LS^l}{N} \quad (2)$$

$$R_{MC} = \frac{\left(N \times \sum_{l=1}^L SS^l - \left(\sum_{l=1}^L LS^l \right)^2 \right)^{\frac{1}{2}}}{(N)^2} \quad (3)$$

Algorithm 1 Create initial model.**Require:** D_{init} : initial training data, K : number of clusters**Ensure:** Model

```

Model  $\leftarrow \phi$ 
 $C \leftarrow \text{Cluster}(D_{\text{init}}, K)$  ▷ clustering on initial data with k-means
for all  $C_i \in C$  do
  for all  $C_i^l \in C_i$  do
     $N^l \leftarrow |C_i^l|$  ▷ total data points of class  $l$ 
     $LS^l \leftarrow \sum_{j=1}^{N^l} X_j$  ▷ linear sum
     $SS^l \leftarrow \sum_{j=1}^{N^l} X_j^2$  ▷ squared sum
     $C_{MC} \leftarrow \frac{\sum_{l=1}^L LS^l}{N}$  ▷ center
     $R_{MC} \leftarrow \frac{\left( N \times \sum_{l=1}^L SS^l - \left( \sum_{l=1}^L LS^l \right)^2 \right)^{\frac{1}{2}}}{(N)^2}$  ▷ radius
  end for
   $W \leftarrow 1$ 
   $W_{ca} \leftarrow 0$ 
   $T \leftarrow 0$ 
   $MC \leftarrow (LS, SS, N, C_{MC}, R_{MC}, W, W_{ca}, T)$  ▷ MC(micro-cluster)
  Model  $\leftarrow \text{Model} \cup MC$ 
end for
return Model

```

where $|C_i^l|$ represents instances with label l in cluster C_i .

3.3. Online active learning strategy

An active labeling strategy is the process of determining whether to query the real label for a given instance. Three active learning strategies are introduced in this framework: random strategy, concept adaptive strategy, and local entropy strategy. The simplest instance selection strategy is the random selection strategy. This approach is grounded in the recognition that in a data flow environment, class distributions can exhibit substantial temporal variations. While conventional active learning algorithms aim to label “significant” instances, they may inadvertently skew the class distribution by favoring one category of instances, thus distorting the true class distribution of labeled examples. The random strategy process is as follows: Initially, generate a randomly distributed variable ξ . Subsequently, assess the magnitude of ξ against the threshold σ . If ξ is not greater than σ , the resulting label is true; otherwise, it is false. Despite its simplicity, the random strategy serves as a means to evaluate the model’s predictive performance across the entire data space without introducing bias. Additionally, only the real labels obtained through the random strategy or initialization phase are pushed into the label sliding window, and the information of the label sliding window provides a basis for subsequent sampling decisions of the concept adaptive strategy.

3.3.1. Uncertainty strategy based on concept adaptive and local entropy

While the random selection strategy can obtain a class distribution that is most similar to the current time’s class distribution, indiscriminately selecting instances with this strategy can significantly waste label predictions and fail to focus on the most important instances and regions in the data distribution. Therefore, building upon the random selection strategy, we propose a concept adaptive strategy. The core idea of concept adaptation strategy is to dynamically capture areas that may suffer from concept drift in an error-driven manner. Specifically, if the misclassification rate of a microcluster exceeds the threshold, we believe that the possibility of concept drift in the microcluster and its domain is high. By assigning higher training weights to these microclusters and their domains, instances in this region are more easily selected. The detailed process is described in Algorithm 2. If W_{ca} is greater than W_θ , we can consider that the current instance is more likely to lean towards the current concept and choose to label the instance. Additionally, if the label of the instance belongs to a minority class, we further increase the W_{ca} of the micro-cluster, giving it a greater chance of being selected in subsequent selection processes. The imbalance indicator of the class y , imb_y , is calculated as the following equation (4). Furthermore, W_{ca} will be updated over time, as shown in equation (5). Through the concept adaptive selection strategy, we assign greater weights to micro-clusters likely to experience concept drift and micro-clusters of minority classes, thereby increasing their chances of being selected in subsequent active learning selections.

$$imb_y = \frac{1}{labelnum_y / ((sizelab - numNull) / C)} \quad (4)$$

where the number of class labels for class y in the label sliding window is represented by $labelnum_y$. The size of the sliding window is represented by $sizelab$, the number of placeholders in the label sliding window is denoted as $numNull$, and C denotes the number of classes in the data stream. When imb_y equals 1, the proportion of instances in class y is at the average level. The higher the level of imbalance, the larger the value of imb_y .

$$W_{ca} = W_{ca} \times 2^{-\lambda_p \times (eT)} \quad (5)$$

Where λ_p is the decay rate of W_{ca} , and eT is the elapsed time from the last update of the micro-cluster

Algorithm 2 ConceptAdaptiveStrategy.

```

1: Input:
2:  $W_{ca}$ : The concept adaptive strategy value,  $imb_y$ : The level of imbalance in class y,  $\theta_{imb}$ : Imbalance threshold for majority class
3: Output:
4: labeling  $\in \{\text{true}, \text{false}\}$ ,  $W_{ca}$ 
5: Process:
6: if  $W_{ca} > W_\theta$  then
7:   labeling = true
8:   if  $imb_y > \theta_{imb}$  then
9:      $W_{ca} = W_{ca} + imb_y$ 
10:   end if
11: else
12:   labeling = false
13: end if

```

▷ when y is a minority class
▷ give more weight to the minority class

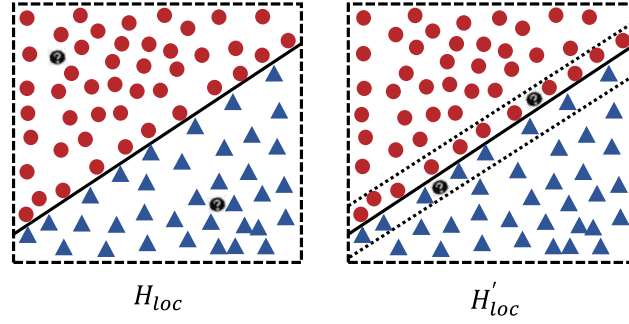


Fig. 3. The location of the instance and its entropy.

In active learning, uncertainty strategies involve seeking the most uncertain points to search for the points with the maximum amount of information. These points are believed to be close to the decision boundary. One method of measuring the uncertainty of an instance is based on marginal measures, which simultaneously consider the maximum posterior probability and the second most likely class label, tending to select instances with the smallest margin between the posterior probabilities of the two most likely class labels. In this paper, we use the local class entropy method to measure the uncertainty of an instance. Since only a small portion of labeled data can be obtained, how to obtain more valuable data is a key issue. Our motivation to use local class entropy comes from an easily understood phenomenon, that is, when an instance is closer to the region near the decision boundary, there are more data label types around the instance than in the region away from the decision boundary. In other words, if the local class entropy of the current instance is high, it means that the instance is more likely to be located near the decision boundary. We will be more likely to choose instances near these regions, as shown in Fig. 3, black circles represent the unlabeled current instances. The specific calculation formula is shown in Equation (6). First, find the K nearest neighbors of instance X and identify the K nearest micro-clusters. Then, calculate the maximum class entropy based on the classes of the K micro-clusters. This class entropy is the local class entropy of instance X. When the local class entropy of the current instance is greater than the local class entropy threshold, the current instance is actively chosen for labeling, as shown in equation (7). In this way, we bias the data chosen by active learning towards the decision boundary, allowing us to approximately depict the dynamic decision boundary in the evolving data stream, thereby achieving the goal of finding the points with the most information.

$$H(x) = \max_k \left(- \sum_{i=1}^C P(x_i) \cdot \log_2(P(x_i)) \right) \quad (6)$$

where the k is nearest neighbors of instance X, C is C classes in K nearest neighbors, and $P(x_i)$ is the probability of class i.

$$\text{labeling}(\text{Local Entropy Strategy}) = \mathbb{I}(H > H_{loc}) \quad (7)$$

where $\mathbb{I}$ is an identity function that outputs 1 if $H > H_{loc}$ is true and otherwise outputs zero.

3.4. Creation of classification model

Traditional micro cluster update methods process a micro-cluster in the following ways: adding an instance to the micro-cluster, merging two micro-clusters, and deleting the micro cluster. This means that if a micro-cluster remains in the model for a long time, its radius will continuously increase. In the presence of minority classes, the continuously increasing radius of the majority class micro cluster makes it easier to capture instances, while due to the infrequent occurrence of minority classes, the update frequency of minority class micro-clusters is very low, making it more difficult to capture minority class instances. Therefore, we use Algorithm 3 to perform more detailed updates based on the location of the instance in the micro clusters. Before that, we use a classification algorithm to get the prediction label *pre_label* of the instance.

Algorithm 3 Creation of classification model.

```

1: Input:
2: DS: data stream
3:  $D_{ic}$ : The location of the instance from the center of the microcluster
4:  $R_{MCi}$ : The radius of the micro-cluster  $MCi$ 
5:  $W_{ca}$ : The concept adaptive weights for micro-cluster  $MCi$ 
6: Output:
7: Model
8: Process:
9: while (DS hasNext()) do
10:    $[pre\_label, MCs] = classify(x, Model)$ 
11:   if (active labeling) then
12:     Get the true label of  $x$ , which is  $x.trueLabel$ 
13:     if  $0 \leq D_{ic} < \frac{R_{MCi}}{2}$  then
14:       if  $pre\_label == x.trueLabel$  then
15:          $W_{MCi} += 1$ 
16:       else
17:          $W_{ca} += 1$ 
18:          $W_{MCi} -= 1$ 
19:         if  $W_{MCi} < 0$  then
20:           Remove  $MCi$ 
21:         end if
22:       end if
23:     else if  $\frac{R_{MCi}}{2} \leq D_{ic} < R_{MCi}$  then
24:       if  $pre\_label == x.trueLabel$  then
25:          $W_{MCi} += 1$ 
26:       else
27:         The  $MCi$  is break into  $MCi'$ ,  $MC_{new}$ 
28:          $W_{MC_{new}} = 2 \times W_{MCi'}$ 
29:       end if
30:     else
31:       Create new MC with  $x.trueLabel$ 
32:     end if
33:      $Model = Model \cup MC$ 
34:   end if
35: end while

```

▷ MC inconsistent with true label

A micro-cluster level lazy learning framework based on cluster structure distance and cluster purity is used to predict the class labels of incoming instances in a data stream. Given a test instance X and a micro cluster (MC), for each class label l , the Cluster Structure Distance (CSD, as shown in equation (6)) refers to the distance from the instance X to the center C_{MC} of micro-cluster MC_i that belongs to label l .

$$CSD(x, MC_i^l) = \text{dist}(x, C_{MC_i^l}) \quad (8)$$

where $\text{dist}(\cdot)$ represents the Euclidean distance from X to the center of MC_i^l . Building upon the CSD distances, we utilize these micro-clusters for lazy learning (i.e., micro-cluster-based k-NN) predictions. First, for a given test instance x , we search for its k nearest micro-clusters $\{MC\}_i^l$, and then the label of x and the corresponding micro-clusters are determined by cluster structure distance and cluster purity.

$$pre_label(x) = \underset{l}{\operatorname{argmax}} \left(\sum_{MC_i \in \{MC\}_i^k} \frac{\text{Pur}(MC_i^l)}{CSD(x, MC_i^l)} \right) \quad (9)$$

where $\text{Pur}(MC_i)$ represents the purity of MC_i^l . $\{MC\}_i^k$ represent k number of nearest micro-clusters. $CSD(x, MC_i^l)$ is the distance from the instance X to the center C_{MC} of micro-cluster MC_i that belongs to label l .

If the real label is requested and if the current instance is within half the radius of the micro-cluster (as in case 1 in Fig. 4), if the predicted label is consistent with the true label, then the weight W of the current micro-cluster increases. Otherwise, the weight W_{ca} of the current micro-cluster decreases. The concept adaptive weights are akin to the weight formula for micro-clusters (see equation (5)), with the distinction that the λ_p for concept adaptive weights is relatively greater than the λ . This implies that concept adaptive weights update more rapidly. Furthermore, concept adaptive weights are associated with minority classes, thus making them more inclined towards the micro-clusters of the most recent concepts and micro-clusters of minority classes. If the current instance is between half the radius and the full radius of the micro-cluster (as in case 2 in Fig. 4), if the predicted label is consistent with the true label, then the weight W of the micro-cluster increases. Otherwise, the original micro-cluster is split into two micro-clusters, namely mc_i' and mc_{new} . The calculation formulas for the new micro-cluster mc_{new} are shown in Equations (10) and (11). Correspondingly, LS' , SS' , R' , and N' of the original MC are updated to $LS - LS^n$, $SS - SS^n$, $R - R^n$ and $N - N^n$. After splitting into two micro-clusters, the new micro-cluster mc_{new} is given a larger concept adaptive weight W_{ca} , because compared to mc_i , the new cluster mc_{new} is better able to represent the current concept. If the current instance is outside the micro-cluster (as in case 3 in the Fig. 4), a new micro-cluster is regenerated. Through the above more detailed updating method for micro-clusters, as

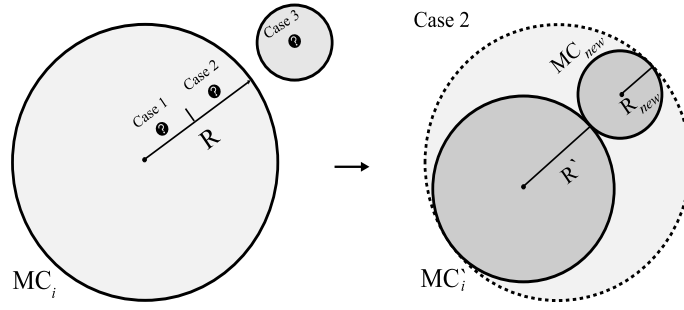


Fig. 4. Different locations of instance in micro cluster.

well as the introduction of the new concept adaptive weight W_{ca} , we enable the model to continuously adapt to concept drift in the evolving data stream, focus more on the current concept, and through the concept adaptive strategy, make the model focus more on the minority class, giving the minority class more opportunities to be selected by the active learning strategy.

$$R_a = IC/R_{MC}, \quad LS^n = R_a * LS, \quad SS^n = R_a^2 * SS \quad (10)$$

$$R^n = \frac{(1 - R_a)}{2} * R_{MC}, \quad N^n = [(1 - R_a) * N] \quad (11)$$

where IC represents the Euclidean distance from input instance to the center of the nearest neighbor micro-cluster. R_{MC} represents the radius of the nearest neighbor micro-cluster. LS^n , SS^n , R^n , and N^n represent the linear sum, the sum of squares, the radius of the new micro-cluster, and the number of instances in the new micro-cluster formed by splitting, respectively.

3.5. Model update

The creation part of the classification model mentioned earlier involves updating micro-clusters, but this part of the micro-cluster update mainly focuses on the active learning micro-cluster update. For updating the entire learning model, first, the weight exponential decay function of micro-clusters is used to delete outdated and low-confidence clusters. The micro-clusters are updated as follows:

$$W_{MC} = W_{MC} \times 2^{-\lambda(eT)} \quad (12)$$

Where λ is the decay rate, and eT is the time elapsed since the last micro-cluster update. When the weight of micro-cluster in the model is less than or approximately equal to 0, the micro-cluster will be removed from the model. Next, merge or delete operations are used to create a new space. If there are no unlabeled clusters in the classification model, the closest unlabeled and labeled clusters are searched for merging. If there are no unlabeled micro-clusters, the model searches for the micro-cluster with the largest number of the same labels. Then, it finds the closest two micro-clusters and judges their distance and maximum radius. If the distance between two micro-clusters is less than the maximum radius between them, the two micro-clusters are merged. Otherwise, the oldest micro-cluster in the model is deleted. Algorithm 4 combined all parts to obtain the LEPID model.

3.6. Complexity analysis

Time complexity The overall time complexity of the LEPID algorithm is primarily determined by three modules: the initialization module, the classification module, and the online update module. Initially, the D_{init} (set as 2000 in our study for all datasets) is partitioned into several distinct sets, each corresponding to a specific class. The K-Means algorithm (with the number of clusters set as k_{init}) is then employed to generate initial micro-clusters for each set. Consequently, the time complexity of the initialization process is $O(L \cdot k_{init} \cdot |X^c|)$, where L is the number of classes, k_{init} is the number of clusters, and $|X^c|$ is the number of instances in class c . During the classification phase, the run-time of KNN classification is $O(maxMC)$. Here, maxMC represents the maximum number of micro-clusters maintained in the model. During the online maintenance phase, there are three primary modules: merging two micro-clusters, deleting a micro-cluster, splitting a micro-cluster. To merge two micro-clusters, we need to search for the closest pair of micro-clusters in the model, so the time complexity is $O(maxMC^2)$. Deleting a microcluster requires finding a microcluster with a weight less than 0 among all microclusters, and the time complexity in the worst case is $O(maxMC)$. The time complexity of splitting a micro-cluster is $O(2 \cdot |X^c|)$. Thus, the overall time complexity of our algorithm is $O(L \cdot k_{init} \cdot |X^c| + maxMC + maxMC^2 + maxMC + 2 \cdot |X^c|) \approx O(L \cdot k_{init} \cdot |X^c| + maxMC^2)$.

Space complexity The space complexity of the model is given as follows: A single MC takes $O(2 \times V_{1 \times d} + 5)$ of space. Hence, the overall space complexity of the model is $O(maxMC * MC)$.

Algorithm 4 Algorithm of LEPID.

```

1: Input:
   DS: data stream, B: labeling budget,  $b_t$ : labeling budget at time  $t$ ,  $l$ : The number of labeled instances
    $\sigma$ : random strategy ratio, C: The number of classes
    $S_{lsw}$ : The size of the label sliding window
    $S_c$ : The size of each sample sliding window
2: Output: Model
3:  $Model \leftarrow createInitialModel(D_{init})$ 
4:  $l \leftarrow 0$ 
5:  $S_c \leftarrow S_{lsw} \times \frac{\xi}{C}$ 
6:  $L[S] \leftarrow \emptyset$ 
7: while (DS.hasNext()) do
8:    $x_t \leftarrow DS.nextInstance()$ 
9:   if  $b_t < B$  then
10:     Generate a uniform random variable  $\xi \in [0, 1]$ 
11:     if  $t < L_{lsw}$  or  $\xi < \sigma$  then
12:       labeling = True
13:        $L[S] \leftarrow \text{true label}$ 
14:     else if (ConceptAdaptiveStrategy) then
15:       labeling = True
16:        $L[S] \leftarrow \text{Null}$ 
17:     else if (LocalEntropyStrategy) then
18:       labeling = True
19:        $L[S] \leftarrow \text{Null}$ 
20:     else
21:       labeling = False
22:     end if
23:   else
24:     labeling = False
25:   end if
26:   if (labeling) then
27:     Request the true label  $y_t$  of instance  $x_t$ 
28:      $l \leftarrow l + 1$ 
29:     Calculate the dynamic multi-class imbalance ratio
30:     Update  $b_t$ 
31:     Update Model with  $(x_t, y_t)$ 
32:   end if
33:    $Model \leftarrow ModelUpdate$ 
34: end while

```

4. Experiments

In this section, we offer a comprehensive exposition of real and synthetic datasets, furnishing detailed descriptions and enumerating the competing algorithms. Ultimately, we present a meticulous analysis of the empirical results derived from our study.

4.1. Data sets

We evaluate the proposed method on synthetic data and six different types of real-world data: Weather, Electricity, EEG, Gas sensor, and Shuttle. Table 1 lists the characteristics of all data sets. The experiments were executed on a computer with a 3.6 GHz CPU and 8 GB RAM. All experiments were repeated ten times with the random seeds set from 1 to 10. The results shown are the average values of ten trials.

4.1.1. Real-world data sets

Here, six real data sets are used to evaluate our proposed algorithm.

Weather¹: This dataset comprises 18,159 instances with 8 attributes, spanning the data collection period from 1949 to 1999. The attributes encompass variables like temperature, pressure, wind speed, and others, all meticulously recorded at the Offutt Air Force Base in Bellevue, Nebraska. The primary objective of this dataset pertains to predicting the occurrence of rainfall on specific days.

Electricity²: This data set consists of 45,312 instances, each characterized by eight distinct features. These examples are categorized into two distinct classes, signifying alterations in electricity prices, with one class denoting an increase and the other a decrease.

EEG³: This dataset consists of a single continuous EEG measurement obtained using the Emotiv EEG Neuroheadset. The measurement lasted for 117 seconds. The eye state was identified using a camera during the EEG recording and was subsequently added manually to the file after analyzing the video frames. It contains 14980 examples with 14 attributes.

¹ <https://users.rowan.edu/~polikar/nse.html>.

² http://mlkd.csd.auth.gr/concept_drift.html.

³ <https://archive.ics.uci.edu/dataset/264/eeeg+eye+state>.

Table 1
Characteristic of datasets.

Dataset	# Ins	# F	# C	Imbalance Ratio	Drift Type
Weather	18159	8	2	3:7	Unknown
Electricity	45312	8	2	19:26	Unknown
EEG	14980	14	2	-	Unknown
GasSensor	13616	128	6	21:21:18:13:13:11	Unknown
Shuttle	58000	10	8	45586:49:171:8903:3267:10:13	Unknown
Coverttype	581012	54	7	77:103:13:1:3:6:7	Unknown
Sea_sudden	50000	3	2	5:1 → 1:5	Sudden
Sea_gradual	50000	3	2	5:1 → 1:5	Gradual
RBF_sudden	50000	21	5	10:5:4:1 → 1:2:2:5	Sudden
RBF_gradual	50000	21	5	10:5:4:1 → 1:2:2:5	Gradual
Agrawal_sudden	50000	9	2	1:9 → 9:1	Sudden
Agrawal_gradual	50000	9	2	1:9 → 9:1	Gradual
HyperPlane	50000	5	2	1:10	Incremental
Wave_sudden	50000	40	3	1:10:40 → 40:1:10	Sudden
Wave_gradual	50000	40	3	1:10:40 → 40:1:10	Gradual

Gas Sensor Array Drift⁴: This dataset originates from the collective readings of 16 distinct chemical sensors, deployed for the purpose of classifying six distinct gases. It encompasses a total of 13,910 instances, which cover a wide range of chemical concentration levels across the spectrum of these six gases.

Shuttle⁵: The data set comprises 58,000 instances, classified into seven distinct categories, with each instance characterized by nine real-valued features. Notably, approximately 2% of the instances are distributed among six of these classes, while one class dominates the data set, accounting for nearly 80% of the total data.

Coverttype⁶: There are 581,012 examples in this data set, and each example has 54 features. This data set describes seven types of forest cover from different geographical areas.

4.1.2. Synthetic data

We adopt the synthetic generators SEA, RandomRBF, and Agrawal provided by the MOA data stream software suite to simulate different types of data streams in the following experiments. The SEA generator was used to generate two streams: SEA_sudden and SEA_gradual. SEA_sudden with a sudden drift and a dynamic imbalance ratio (IR) 5:1 → 1:5; SEA_gradual with a gradual drift and a dynamic IR 5:1 → 1:5. Similarly, Agrawal_sudden and Agrawal_gradual represent sudden concept drift data streams with a dynamic IR 1:9 → 9:1 and gradual concept drift data streams with a dynamic IR 1:9 → 9:1, respectively. In addition, we also synthesize the RandomRBF datasets to explore the effect of the algorithm on the multiclass imbalanced data stream. RBF_sudden involves sudden drifts and a dynamic IR of 10:5:4:1 → 1:2:2:5; RBF_gradual involves gradual drifts and a dynamic IR of 10:5:4:1 → 1:2:2:5. The hyperplane generator simulated incremental drifts with five attributes, two classes, a constant IR of 1:10, and contains 5% noise. The Waveform generator is used to generate two streams: Wave_sudden and Wave_gradual. They have a total of 40 attributes, of which 21 are numeric attributes containing noise and 19 are irrelevant attributes. Wave_sudden and Wave_gradual represent sudden concept drift data streams with a dynamic IR 1:10:40 → 40:1:10 and gradual concept drift data streams with a dynamic IR 1:10:40 → 40:1:10, respectively. Table 1 shows a brief description of the datasets.

4.2. Evaluation metrics

Accuracy, a conventional metric, is insufficient for providing a comprehensive assessment of a classifier's performance on imbalanced datasets. In contrast, the area under the curve (AUC), F1-score, G-mean, recall, and Q-statistic offer more objective measures for evaluating imbalanced classification methods. In this paper, we utilize G-mean, recall and F1-score to assess classifier performance. Specifically, G-mean evaluates the balance between accurately classifying both minority and majority samples, while recall gauges the classification performance of the minority class.

$$G - mean = \sqrt{Recall \times Specificity} \quad (13)$$

4.3. Comparison method

To validate the effectiveness of the LEPID algorithm, we compared it with well-known and commonly used state-of-the-art algorithms. We not only compared the latest active learning methods but also the latest ensemble learning methods and approaches specifically designed for imbalanced data streams. The algorithms used for these comparisons are briefly described below. The code

⁴ <https://archive.ics.uci.edu/dataset>.

⁵ <https://archive.ics.uci.edu/dataset/148/statlog+shuttle>.

⁶ <https://moa.cms.waikato.ac.nz/datasets/>.

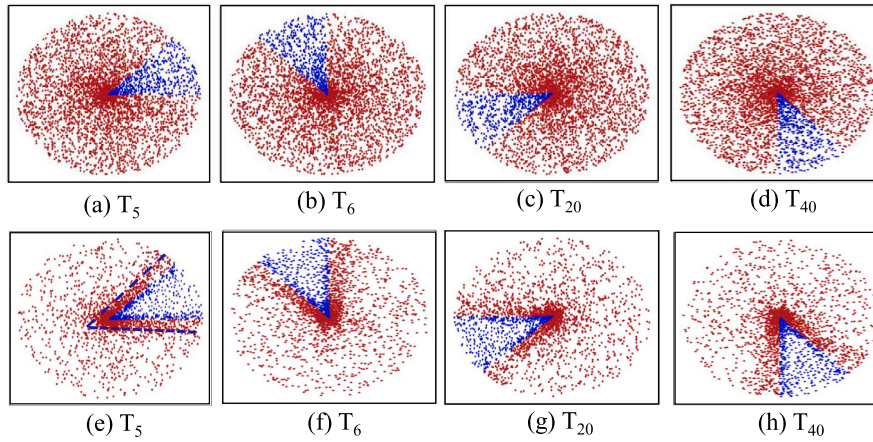


Fig. 5. Performance of the LEPID algorithm on the Imb_S dataset with abrupt concept drift. (a) - (d) show the real distributions of the Imb_S dataset with abrupt concept drift corresponding to the time units, respectively. (e)-(h) show the distribution of the Imb_S that the LEPID algorithm actively chooses to label with a limited labeling budget (20%) for the corresponding time units.

for LEPID is available at the website⁷. And the code for most of the above-mentioned contrast algorithms can be found on this website⁸.

OOB [33]. OOB is an extension of the Online Bagging algorithm. The oversampling ratio is used as λ in the Poisson distribution of online bagging.

UOB [33]. Diverging from the OOB, the UOB incorporates an undersampling technique alongside online bagging.

VFC-SMOTE [34]. This algorithm is an improvement on Smote's Borderline-Smote, saves the samples in a sketch that incorporates Adwin, and uses the summaries contained in the sketch to introduce new synthetic samples.

ROSE [35]. ROSE achieves the capability to handle both standard and imbalanced data streams without the need for switching between these modes by using balanced buffers per class to store instances for training new classifiers.

KUE [36]. KUE offered a hybrid architecture, combining the advantages of online adaptation of base classifiers and block-based updating of the ensemble line-up.

LB [37]. A leveraging bagging method that uses randomization on the weights of the instances of the input stream to improve the accuracy of the ensemble classifier.

ARF [38]. A concept drift detection strategy is integrated with the random forest tree, with the objective of substituting an outdated tree with a fresh one upon the occurrence of a drift. The default drift detector employed is ADWIN.

ARFR [39]. The ARFR approach alters the λ by inserting weights, resulting in a decreased likelihood of presenting majority class instances to the ARF trees while significantly increasing the chances for minority instances.

CALMID [40]. CALMID is devised to address the challenges of multi-class imbalance and concept-drifting data streams. The adaptive window algorithm is employed for detecting the presence of concept drift. Upon the detection of concept drift, a class-balanced training sample sequence is generated by utilizing the stored samples within the sliding window, enabling the training of a new base classifier that is subsequently incorporated into the ensemble classifier.

For a fair comparison, all comparative ensemble learning uses the Hoeffding tree as its base classifier, except for ARF and ARFR, which utilize the specially designed ARF Hoeffding tree. The default value for the size of the label sliding window, *szelab* is set to 500 for easy comparison with CALMID. The default value of the labeling cost budget *b* is set to 10%.

4.4. Proof of concept

We employ two-dimensional synthetic data streams to assess the performance of LEPID and showcase its favorable characteristics.

4.4.1. Concept modeling

Initially, we assessed the capability of LEPID to capture the present concept and adapt to changes in decision boundaries. Fig. 5 (a)–(d) and Fig. 6 (a)–(f) show the real distributions of the synthetic data stream with gradual concept drift and abrupt concept drift corresponding to the time units, respectively. Moreover, the ratio of the majority class to the minority class in the synthetic data is 7:1. Imb_S has 40000 instances, and it is divided into 40 time units. Imb_G has 60000 instances divided into 60-time units. There are 1000 instances per time unit. Fig. 5 (e)–(h) as well as (g)–(l) of Fig. 6 represent the distributions where the LEPID algorithm actively selects instances from Imb_S and Imb_G with a limited label budget of 20%, respectively. From the figures, we can see that the minority class instances of Imb_S and Imb_G drift counterclockwise over time. If we only randomly select instances for labeling,

⁷ <https://github.com/Jolly-w/LEPID>.

⁸ <https://github.com/canoalberto/budgetbasedoversampling>.

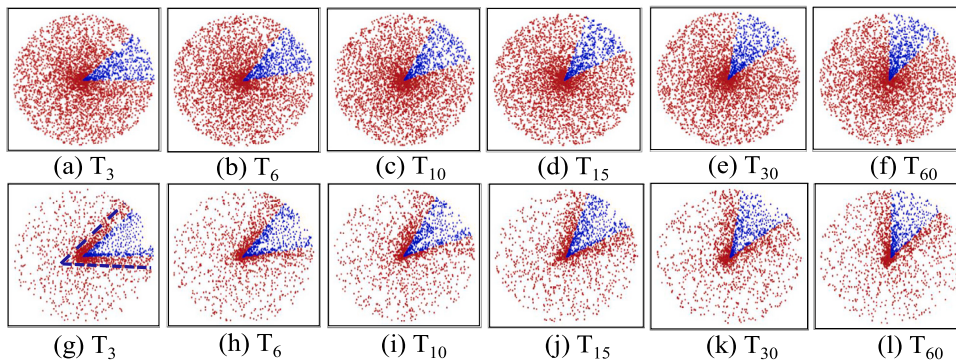


Fig. 6. Performance of the LEPID algorithm on the Imb_G dataset with gradual concept drift. (a) - (f) show the real distributions of the Imb_G dataset with gradual concept drift corresponding to the time units, respectively. (g)-(l) denote the distribution of the Imb_G that the LEPID algorithm actively chooses to label with a limited labeling budget (20%) for the corresponding time units.

Table 2
The ablation study of LEPID under different active learning strategies.

	Imb_S			RBF_gradual		
	Gmean	Recall	F1	Gmean	Recall	F1
LEPID-CA	84.70	74.80	83.35	84.58	77.24	80.24
LEPID-LE	87.89	80.34	74.15	86.29	80.26	82.58
LEPID-CA-LE	83.75	74.15	82.99	83.81	78.49	83.22
LEPID	88.52	82.99	87.28	88.59	83.02	83.47

although the distribution of the extracted instances is very similar to the real distribution, this method of selecting instances without focus will waste a lot of labeling budget. In the LEPID algorithm, while preserving the real distribution of the randomly labeled data, our active learning strategy is more inclined to the instances of the current concept that have concept drift, the instances close to the decision boundary that are difficult to classify, and the instances of the minority class. As shown in Fig. 5 (e) and Fig. 6 (g), the density in the small sector of the blue dashed line is significantly higher than the region next to it, which indicates that the LEPID algorithm can pay more attention to these important regions. Over time, when sudden concept drifts or gradual concept drifts occur, the LEPID algorithm still focuses on these areas, as shown in Fig. 5 (f)-(h) and Fig. 6 (h)-(l), which suggests that LEPID can dynamically adapt to the concept drift and the change of decision boundary in the evolving data stream.

4.5. Ablation study

To investigate the contributions of each component in LEPID, we conducted an ablation study on the Imb_S dataset. The first algorithm compared is LEPID-CA, representing LEPID with the concept adaptive strategy removed. After removing the local entropy strategy, we obtained LEPID-LE. Furthermore, by eliminating both the concept adaptive strategy and the local entropy strategy, we formed LEPID-CA-LE, representing LEPID with only a random strategy. The last one is the complete LEPID algorithm. The changes in algorithm performance metrics are shown in Table 2. We can observe that removing the concept adaptive strategy and the local entropy strategy leads to a decrease in algorithm performance. Particularly when both concept adaptive strategy and the local entropy strategy are removed and only the random sampling strategy is used for active learning, the algorithm's performance is the lowest. The complete algorithm exhibits the highest performance.

4.6. G-mean results

Table 3 presents the average G-mean values for various comparative methods across both synthetic and real-world data streams when the budget is 10%. LEPID achieves the best G-mean indicator values on all the data streams except Electricity and HyperPlane. On the Electricity and HyperPlane datasets, the ROSE exhibits the highest G-mean. As observed in Fig. 7, LEPID consistently produces stable g-mean values across synthetic datasets experiencing sudden changes in data distribution over time. Ensemble learning methods such as OOB and UOB exhibit similar performances on three datasets. The G-mean curve of VFC-SMOTE, an imbalanced learning method, differs markedly from most algorithms. After a sudden drift at the 10,000th point, most algorithms experience a rapid decline in G-mean, which gradually improves over time. This trend is particularly pronounced in the RBF_sudden dataset. It's noted that many algorithms initially have a G-mean of zero due to the failure to learning minority classes at the onset of the data stream. However, as the model learn about minority classes over time, G-mean generally rises above zero. LEPID's G-mean is consistently non-zero, attributed to its model initialization in the LEPID algorithm. Furthermore, as a micro-cluster-based online learning method, LEPID exhibits less fluctuation in G-mean throughout the learning process compared to other algorithms. With the sudden drift of the three datasets, the G-mean of LEPID was gradually reduced, and gradually increased as the new concept was

Table 3
G-mean results (%) of various classification methods.

DataSet	OOB	UOB	VFC-SMOTE	ROSE	KUE	LB	ARF	ARFR	CALMID	Ours
Weather	68.45	66.42	68.11	70.44	59.75	62.78	66.02	70.72	67.10	71.42
Electricity	76.07	73.30	69.72	79.57	68.49	78.69	78.31	78.10	78.35	69.44
EEG	61.72	64.93	69.07	74.06	69.73	75.59	69.01	65.54	77.56	80.91
GasSensor	77.24	78.45	76.33	77.16	65.21	77.82	78.12	82.37	85.72	91.86
Shuttle	77.64	76.52	77.51	89.58	85.50	36.21	58.48	59.07	46.76	99.06
Covtype	78.61	73.78	76.03	83.78	77.46	80.02	78.47	79.24	80.34	86.30
Sea_sudden	76.47	80.36	67.54	78.31	66.67	65.60	72.01	73.70	77.23	83.77
Sea_gradual	74.72	75.93	59.95	74.22	75.26	76.68	73.35	74.91	73.57	84.24
RBF_sudden	63.29	46.05	83.71	80.45	57.73	78.76	81.23	81.95	78.07	88.52
RBF_gradual	70.12	48.07	83.25	83.36	68.83	75.79	83.25	83.65	73.75	88.59
Agrawal_sudden	74.98	76.37	45.64	73.61	71.54	69.44	57.73	70.16	76.37	82.37
Agrawal_gradual	75.25	77.87	29.11	76.02	68.84	65.84	57.04	63.61	50.75	82.45
HyperPlane	76.76	75.35	74.23	78.41	66.13	14.68	35.32	36.24	17.52	61.23
Wave_sudden	75.04	76.15	73.51	80.44	72.54	73.99	78.66	79.01	76.83	87.17
Wave_gradual	73.82	75.73	76.23	77.62	70.29	76.60	77.96	77.45	78.66	87.10

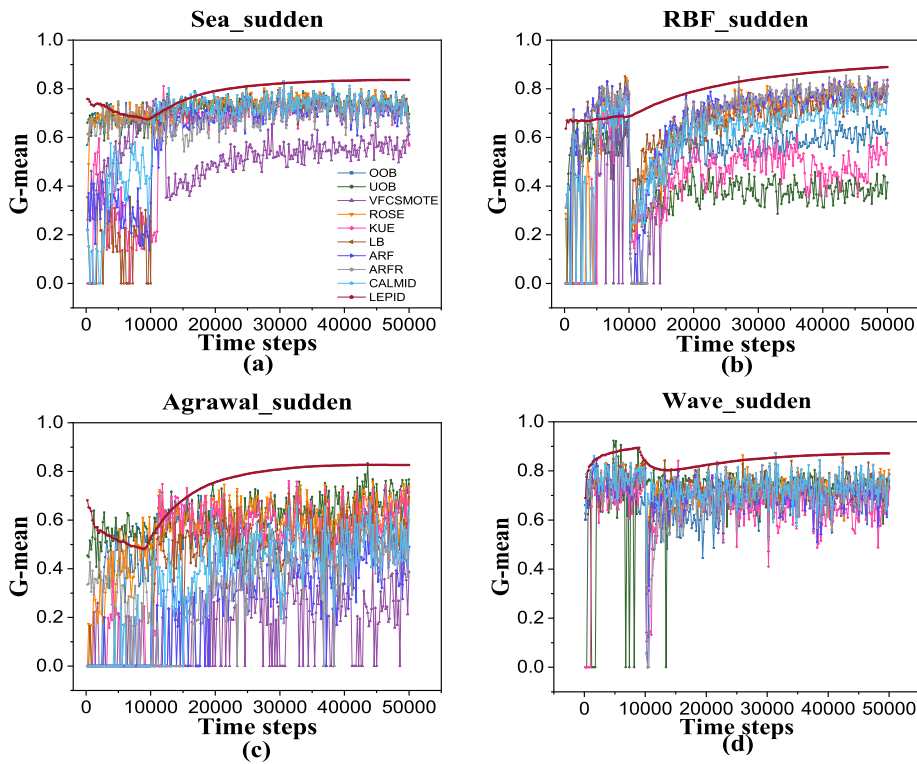


Fig. 7. Prequential G-mean scores for all comparative algorithms are recorded at each time step in the context of data streams with sudden drift. (a) Sea_sudden. (b) RBF_sudden. (c) Agrawal_sudden. (d) Wave_sudden.

adapted, eventually stabilizing. Another noteworthy detail is that, in the process of comparing the experimental results of Fig. 7 and Fig. 8, it can be observed that the LEPID algorithm performs slightly differently on synthetic datasets with different types of concept drift (such as Agrawal_sudden and Agrawal_gradual, Wave_sudden and Wave_gradual). Specifically, before and after abrupt concept drift occurs, the Gmean value of the LEPID algorithm changes abruptly (such as Agrawal_sudden and Wave_sudden); while before and after gradual concept drift occurs, the Gmean value of the LEPID algorithm changes more gradually (such as Agrawal_gradual and Wave_gradual). This corresponds to the type of concept drift. This is because, in abrupt concept drift, the model needs to quickly adapt to the new distribution, resulting in sharp changes in the Gmean value. In gradual drift, the model has enough time to adjust gradually, so the Gmean value changes gently. On noisy data sets Wave_sudden and Wave_gradual, the Gmean value of most algorithms is unstable and frequently equals to 0. In contrast, LEPID consistently maintains a high Gmean value. This stability is due to LEPID's dynamic maintenance of various classes of microclusters, especially those of minority classes. Because the LEPID algorithm pays more attention to the minority class, these microclusters can persist in the model for a long time. This sustained presence enhances the model's classification performance for the minority class. At the same time, since noisy data cannot exist stably in

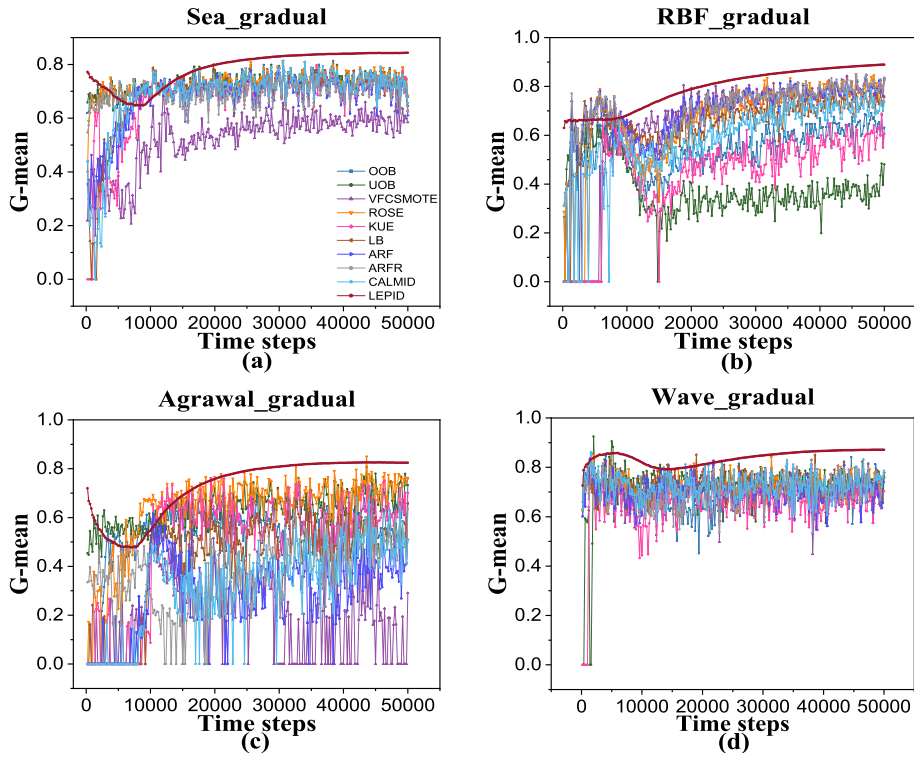


Fig. 8. Prequential G-mean scores for all comparative algorithms are recorded at each time step in the context of data streams with gradual drift. (a) Sea_gradual. (b) RBF_gradual. (c) Agrawal_gradual. (d) Wave_gradual.

Table 4

Recall results (%) of various classification methods.

Dataset	OOB	UOB	VFC-SMOTE	ROSE	KUE	LB	ARF	ARFR	CALMID	Ours
Weather	69.71	68.21	66.47	52.94	59.14	54.90	39.21	65.78	54.91	77.82
Electricity	74.90	71.35	68.95	78.46	69.45	78.96	78.82	76.28	78.05	71.11
EEG	58.12	62.93	67.62	72.85	68.83	74.14	67.62	65.86	66.67	81.51
GasSensor	74.37	76.28	74.26	72.49	68.57	84.75	83.28	83.56	82.31	86.73
Shuttle	74.21	73.42	72.48	72.48	75.24	52.36	55.24	55.41	61.28	76.45
Covtype	69.46	68.64	79.52	79.17	75.38	78.53	74.13	77.90	82.16	87.85
Sea_sudden	73.56	74.36	68.25	67.43	58.51	76.67	75.37	81.63	83.84	85.61
Sea_Gradual	76.51	75.95	62.05	80.39	73.63	73.63	74.76	76.63	77.72	86.07
RBF_sudden	68.73	55.63	82.87	81.65	64.14	79.01	81.03	82.78	79.74	82.99
RBF_gradual	70.13	59.14	81.90	82.42	67.34	79.83	82.85	81.98	76.25	83.22
Agrawal_sudden	75.16	75.02	50.83	75.05	70.83	69.67	60.34	72.70	78.33	82.68
Agrawal_gradual	76.03	76.47	47.65	80.05	68.03	70.07	61.01	67.16	67.19	83.75
HyperPlane	82.31	82.45	76.51	73.58	67.24	35.64	32.67	34.83	37.52	89.36
Wave_sudden	78.21	63.14	74.56	79.25	68.63	78.69	74.58	79.62	80.35	82.69
Wave_gradual	78.68	80.25	78.34	77.38	68.43	69.18	66.34	75.32	76.48	82.95

the model (can not participate in the correct classification, the weight is constantly reduced), LEPID also has a good classification performance in the case of noisy data.

4.7. Recall results

As shown in Table 4, LEPID achieves top-tier recall rates on most data streams when the budget is 10%, indicating that it can not only distinguish minority class instances but also avoid misclassifying other instances. Although LEPID did not secure the top position on the Electricity dataset, its performance still surpasses that of the majority of algorithms. Moreover, whether on abruptly drifting data streams or gradually drifting ones, LEPID maintains high recall rates, with superior performance on gradual drift compared to abrupt drift. This suggests LEPID's capability to adapt to both types of concept drift. The superior recall rate of LEPID can be attributed to the following key factors: During the model initialization phase, it aggregates micro-clusters composed of minority class instances. In the evolution of subsequent data streams, these minority class micro-clusters are more effectively identified and processed by the LEPID algorithm because it assigns higher concept adaptation weights W_{ca} to minority class micro-clusters. Compared to some

Table 5
F1-score results (%) of various classification methods.

Dataset	OOB	UOB	VFC-SMOTE	ROSE	KUE	LB	ARF	ARFR	CALMID	Ours
Weather	71.42	70.19	71.48	73.25	68.56	69.56	73.67	71.45	74.42	75.90
Electricity	77.34	75.62	72.35	81.27	71.86	79.45	79.39	80.51	78.47	70.82
EEG	73.28	75.97	76.82	77.58	72.74	78.46	70.67	79.57	78.83	80.45
GasSensor	80.57	81.79	79.68	82.46	70.58	79.82	80.46	83.52	85.84	87.12
Shuttle	79.47	79.36	78.38	93.48	89.69	60.48	72.46	74.62	65.74	99.38
Covtype	80.26	78.57	79.46	84.63	81.32	82.41	79.36	80.31	83.68	88.52
Sea_sudden	78.28	81.62	73.27	80.42	72.45	71.59	78.33	78.94	80.24	85.89
Sea_Gradual	77.61	78.44	64.20	78.45	79.77	80.23	78.46	79.41	79.22	86.23
RBF_sudden	70.20	60.21	80.21	81.27	63.72	80.23	82.54	82.68	79.86	83.02
RBF_gradual	77.23	79.81	81.34	81.39	76.47	78.43	80.67	80.42	78.25	82.76
Agrawal_sudden	78.27	78.32	60.35	78.46	76.83	75.35	63.24	75.90	79.70	82.90
Agrawal_gradual	79.54	79.61	59.62	79.30	75.39	72.47	62.95	72.61	74.25	84.55
HyperPlane	83.41	84.12	74.61	64.24	78.64	82.62	82.51	83.56	84.51	88.14
Wave_sudden	75.93	78.15	76.17	81.24	76.36	75.88	80.20	80.37	81.06	84.24
Wave_gradual	75.44	79.71	76.28	75.16	73.99	76.63	79.52	78.14	82.80	84.27

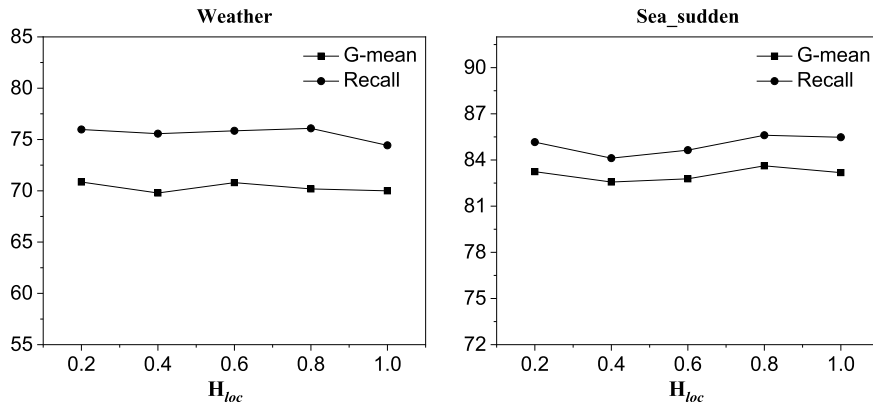


Fig. 9. The sensitivity analysis with H_{loc} on the Weather, Sea_sudden data sets.

online ensemble sampling methods without concept drift handling mechanisms (such as OOB and UOB), the LEPID algorithm, by combining dynamic micro-cluster concept modeling and concept drift adaptation strategies, can dynamically capture data in the data stream that is prone to concept drift. This data is usually more important and valuable, thus increasing the model's sensitivity and response speed to these critical areas. In contrast to some supervised learning algorithms like LB and ARF, the LEPID algorithm, despite acquiring only a small amount of labeled data, employs three active learning strategies to select instances that are more valuable for model training. It obtains the true labels of these instances and promptly updates the model. By dynamically adjusting the decision boundary, the algorithm can better adapt to changes in the data stream, thereby achieving a higher recall value.

4.8. F1-score results

As shown in Table 5, with the exception of the Electricity dataset, the LEPID algorithm outperforms other competing algorithms in terms of F1 score. This advantage can be attributed to the following factors: First, the concept of dynamic micro-cluster modeling allows LEPID to promptly track areas where concept drift occurs. Secondly, LEPID pays more attention to minority class microclusters, which often contain critical information for making precise classifications. By focusing on these key data points, LEPID effectively improves recall without sacrificing precision, thereby achieving a higher F1 score. In contrast, other algorithms often fail to effectively balance the trade-off between precision and recall, particularly in imbalanced or evolving data streams. This results in either a reduction in precision due to overgeneralization or a decrease in recall if minority classes are overlooked. Conversely, LEPID's balanced approach ensures optimal performance across various data conditions, resulting in superior F1 scores.

4.9. Parameter sensitivity analysis

To analyze the effect of different parameters against the classification performance, we have conducted some experiments on the Weather and Sea_sudden datasets with different hyperparameter combinations of H_{loc} , maxMC, sizelab, Budget.

- H_{loc} The local class entropy, H_{loc} , represents the level of confusion among micro-clusters labels near a given instance. As shown in Fig. 9, When H_{loc} has a low value (e.g., 0.1), it indicates that micro-clusters near the instance are predominantly of the same

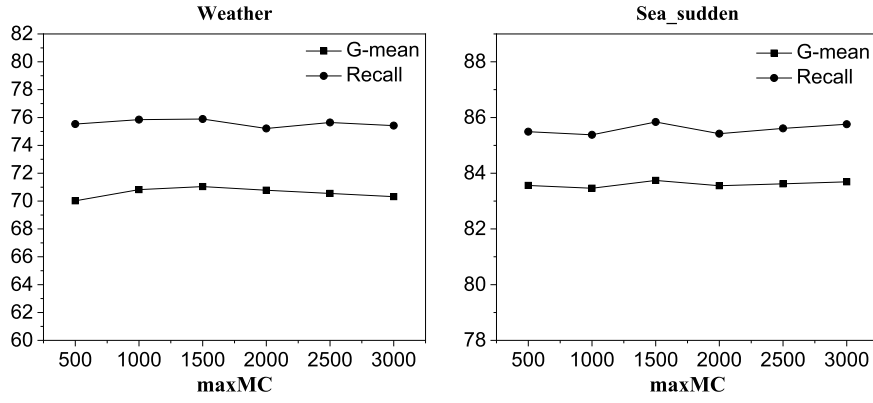


Fig. 10. The sensitivity analysis with maxMC on the Weather, Sea_sudden data sets.

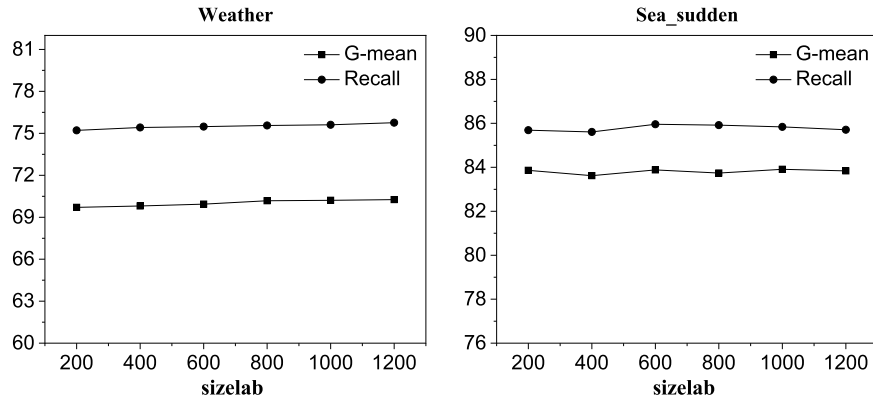


Fig. 11. The sensitivity analysis with sizelab on the Weather, Sea_sudden data sets.

label. Conversely, when H_{loc} has a high value (e.g., 0.8), it suggests that there are multiple labels among the micro-clusters near the instance. As shown in Fig. 9, we observe that the model performs poorly when H_{loc} is set to a small value. On the other hand, when H_{loc} is set to a slightly larger value, the model performs better. This doesn't imply that a larger H_{loc} is always better; considering all factors, an H_{loc} value of 0.8 is deemed an appropriate choice.

- **maxMC** It represents the maximum number of micro-clusters that can be stored in the model to capture the current concept. The larger the maxMC, the more micro-clusters can be stored, providing more comprehensive local conceptual information. However, as shown in Fig. 10, an excessive number of micro-clusters does not significantly enhance the overall performance of the model and increases computational complexity. Therefore, an appropriate choice for maxMC, such as maxMC = 2000, may be a good option.
- **sizelab** Which represents the size of the label sliding window. From Fig. 11, we can see that the variation in sizelab has a limited impact on the model's performance, indicating that LEPID is not highly sensitive to this parameter.
- **Budget** The final parameter under consideration is the labeling cost budget, which is a crucial determinant of the overall labeling expense. This study conducted experiments on *Sea_sudden* datasets, utilizing varying budget ratios of 0.01, 0.02, 0.04, 0.06, 0.08, and 0.1. As illustrated in Fig. 12, a notable observation is that the slope of LEPID is not large compared to other algorithms; that is, as the budget decreases, the Gmean and Recall of LEPID does not change much. This shows that even in the case of sparse labels, LEPID still has good classification performance.

5. Statistical analysis

We further use Friedman rank-sum test to check the statistical significance between competing algorithms. For this purpose, we define the null hypothesis as "no significant difference among comparing algorithms." If the null hypothesis is rejected, we further used the Nemenyi post-hoc test to find these differences. The Friedman rank sum test is a non-parametric statistical test that we perform for Tables 3, 4 and 5 with a 95% confidence level and obtain p-values = 2.89e-06, 8.86e-06 and = 4.88e-07 for all datasets, respectively. The resulting P-value is less than the standard threshold (0.05), which confirms that our method is statistically different from other competing algorithms. Based on the rejection of the null hypothesis, we applied the Nemenyi post-hoc test to build a critical difference graph (see Figs. 13, 14, and 15) to identify the comparative performance between algorithms on G-mean, Recall, and F1 score metrics. Our proposed method has always ranked first, showcasing its robustness and effectiveness across

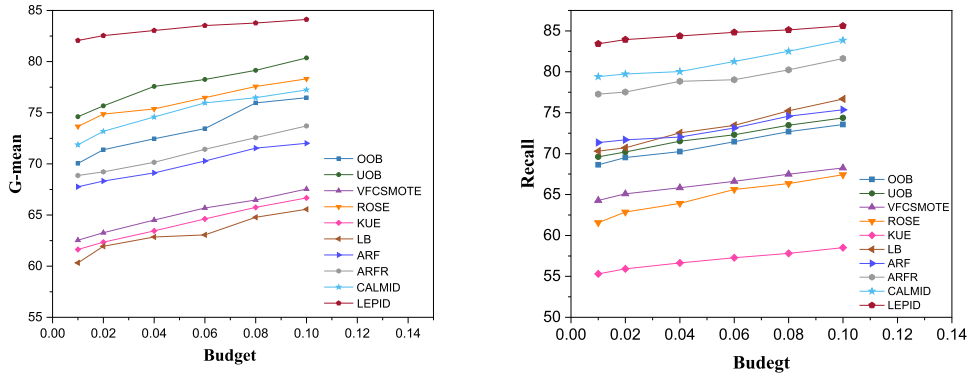


Fig. 12. The sensitivity analysis with Budget and Recall on the Sea_sudden data sets.

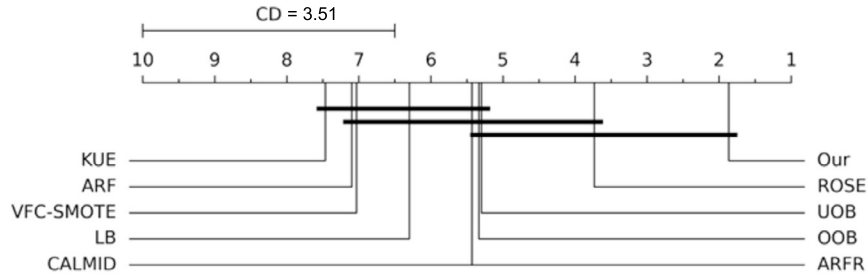


Fig. 13. Nemenyi test with 95% confidence level on G-mean.

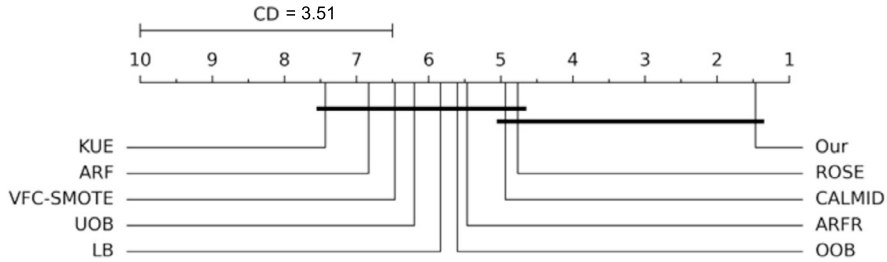


Fig. 14. Nemenyi test with 95% confidence level on Recall.

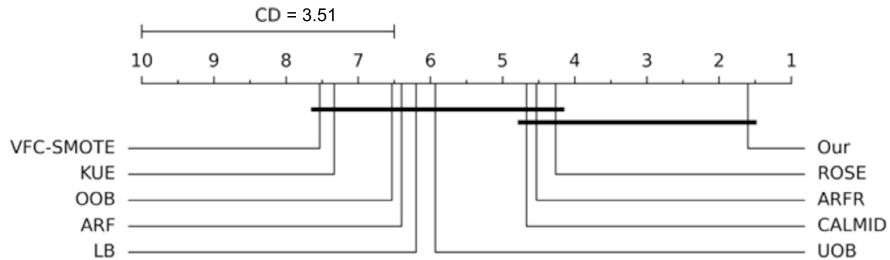


Fig. 15. Nemenyi test with 95% confidence level on F1-score.

these evaluation metrics. Specifically, for the G-mean metric (Fig. 13), our method ranked first, indicating comparable performance to ROSE and UOB while outperforming KUE, ARF, and VFC-SMOTE. For the recall metric (Fig. 14), our approach also attained the top rank, with no significant statistical differences between our method and the continuous learning algorithm ROSE and the active learning algorithm CALMID method, exceeding KUE, ARF and VFC-SMOTE. While ROSE benefits from combining backup ensembles, our algorithm achieves a higher recall rate by paying more attention to minority class regions and concept drift regions and dynamically updating the model in a timely manner. Finally, in terms of the F1-score (Fig. 15), our method maintains the leading position, demonstrating similar performance to ROSE and CALMID and outperforming algorithms such as KUE, OOB, and LB.

6. Conclusion and future works

This paper proposed LEPID, an online active learning framework for data streams with coexisting multiclass imbalances and concept drift. In comparison with existing algorithms, it is able to query the most valuable instances under sparser labels in the current concept. This significant improvement is achieved by combining dynamic micro-cluster concept modeling with micro-cluster-based local entropy modeling, which tends to dynamically allocate higher training weights to the more difficult-to-distinguish and minority class samples for the current concept. Extensive experiments on synthetic and real-world datasets further demonstrate its superiority over many state-of-the-art methods.

The limitation of the LEPID algorithm is its failure to address the class overlap problem, where examples from different classes are located in the same region of the data space, thus compromising the definition of clear decision boundaries. In imbalanced domains, this problem is even more exacerbated, since the minority examples that exist may be located in these overlapping regions. The class overlap problem in static scenarios has been widely studied, however, there are few studies on dealing with the class overlap problem in data stream scenarios. Future research should focus on integrating existing research on class overlap with data stream scenarios to more effectively tackle this challenge in imbalanced data streams.

CRedit authorship contribution statement

Zhonglin Wu: Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Formal analysis, Conceptualization. **Hongliang Wang:** Writing – review & editing, Validation. **Jingxia Guo:** Writing – review & editing, Visualization. **Qinli Yang:** Writing – review & editing, Resources. **Junming Shao:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work is supported by the Municipal Government of Quzhou under Grant No. 2023D016, Fundamental Research Funds for the Central Universities (ZYGX2019Z014), Guangdong Basic and Applied Basic Research Foundation (2024A1515011634), Shenzhen Science and Technology Program (JCYJ20230807120008016), National Natural Science Foundation of China (61976044, 52079026), Chengdu Science and Technology Project (2021-YF08-00127-GX).

References

- [1] Z. Liu, Z. Miao, X. Zhan, J. Wang, B. Gong, S.X. Yu, Large-scale long-tailed recognition in an open world, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 2537–2546.
- [2] H. Li, T. Zhao, A dynamic similarity weighted evolving fuzzy system for concept drift of data streams, *Inf. Sci.* (2024) 120062.
- [3] Y. Chen, X. Yang, H.-L. Dai, Cost-sensitive continuous ensemble kernel learning for imbalanced data streams with concept drift, *Knowl.-Based Syst.* 284 (2024) 111272.
- [4] W.-C. Lin, C.-F. Tsai, Y.-H. Hu, J.-S. Jhang, Clustering-based undersampling in class-imbalanced data, *Inf. Sci.* 409 (2017) 17–26.
- [5] S. Wang, L.L. Minku, X. Yao, Resampling-based ensemble methods for online class imbalance learning, *IEEE Trans. Knowl. Data Eng.* 27 (2014) 1356–1368.
- [6] L. Korycki, B. Krawczyk, Online oversampling for sparsely labeled imbalanced and non-stationary data streams, in: *2020 International Joint Conference on Neural Networks (IJCNN)*, IEEE, 2020, pp. 1–8.
- [7] S. Liu, S. Xue, J. Wu, C. Zhou, J. Yang, Z. Li, J. Cao, Online active learning for drifting data streams, *IEEE Trans. Neural Netw. Learn. Syst.* (2021).
- [8] Z. Liu, X. Zhang, B. Jiang, Active learning with fairness-aware clustering for fair classification considering multiple sensitive attributes, *Inf. Sci.* 647 (2023) 119521.
- [9] J. Shan, H. Zhang, W. Liu, Q. Liu, Online active learning ensemble framework for drifted data streams, *IEEE Trans. Neural Netw. Learn. Syst.* 30 (2018) 486–498.
- [10] S.U. Din, A. Ullah, C.B. Mawuli, Q. Yang, J. Shao, A reliable adaptive prototype-based learning for evolving data streams with limited labels, *Inf. Process. Manag.* 61 (2024) 103532.
- [11] J. Shao, F. Huang, Q. Yang, G. Luo, Robust prototype-based learning on data streams, *IEEE Trans. Knowl. Data Eng.* 30 (2017) 978–991.
- [12] S.U. Din, J. Shao, Exploiting evolving micro-clusters for data stream classification with emerging class detection, *Inf. Sci.* 507 (2020) 404–420.
- [13] A. Fernández, S. García, M. Galar, R.C. Prati, B. Krawczyk, F. Herrera, A. Fernández, S. García, M. Galar, R.C. Prati, et al., Learning from imbalanced data streams, in: *Learning from Imbalanced Data Sets*, 2018, pp. 279–303.
- [14] D. Brzezinski, J. Stefanowski, Prequential auc: properties of the area under the roc curve for data streams with concept drift, *Knowl. Inf. Syst.* 52 (2017) 531–562.
- [15] S. Yu, Z. Abraham, Concept drift detection with hierarchical hypothesis testing, in: *Proceedings of the 2017 SIAM International Conference on Data Mining*, SIAM, 2017, pp. 768–776.
- [16] K. Wu, A. Edwards, W. Fan, J. Gao, K. Zhang, Classifying imbalanced data streams via dynamic feature group weighting with importance sampling, in: *Proceedings of the 2014 SIAM International Conference on Data Mining*, SIAM, 2014, pp. 722–730.
- [17] B. Jiao, Y. Guo, D. Gong, Q. Chen, Dynamic ensemble selection for imbalanced data streams with concept drift, *IEEE Trans. Neural Netw. Learn. Syst.* (2022).
- [18] Y. Lu, Y.-m. Cheung, Y.Y. Tang, Dynamic weighted majority for incremental learning of imbalanced data streams with concept drift, in: *IJCAI*, 2017, pp. 2393–2399.

- [19] Y. Lu, Y.-M. Cheung, Y.Y. Tang, Adaptive chunk-based dynamic weighted majority for imbalanced data streams with concept drift, *IEEE Trans. Neural Netw. Learn. Syst.* 31 (2019) 2764–2778.
- [20] G. Aguiar, B. Krawczyk, A. Cano, A survey on learning from imbalanced data streams: taxonomy, challenges, empirical study, and reproducible experimental framework, *Mach. Learn.* (2023) 1–79.
- [21] S. Wang, L.L. Minku, X. Yao, A learning framework for online class imbalance learning, in: 2013 IEEE Symposium on Computational Intelligence and Ensemble Learning (CIEL), IEEE, 2013, pp. 36–45.
- [22] S. Wang, L.L. Minku, X. Yao, Dealing with multiple classes in online class imbalance learning, in: *IJCAI*, 2016, pp. 2118–2124.
- [23] L.E.B. Ferreira, H.M. Gomes, A. Bifet, L.S. Oliveira, Adaptive random forests with resampling for imbalanced data streams, in: 2019 International Joint Conference on Neural Networks (IJCNN), IEEE, 2019, pp. 1–6.
- [24] I. Žliobaitė, A. Bifet, B. Pfahringer, G. Holmes, Active learning with drifting streaming data, *IEEE Trans. Neural Netw. Learn. Syst.* 25 (2013) 27–39.
- [25] G.J. Aguiar, A. Cano, Dynamic budget allocation for sparsely labeled drifting data streams, *Inf. Sci.* 654 (2024) 119821.
- [26] X. Zhu, P. Zhang, X. Lin, Y. Shi, Active learning from stream data using optimal weight classifier ensemble, *IEEE Trans. Syst. Man Cybern., Part B, Cybern.* 40 (2010) 1607–1621.
- [27] I. Žliobaitė, A. Bifet, B. Pfahringer, G. Holmes, Active learning with drifting streaming data, *IEEE Trans. Neural Netw. Learn. Syst.* 25 (2013) 27–39.
- [28] W. Xu, F. Zhao, Z. Lu, Active learning over evolving data streams using paired ensemble framework, in: 2016 Eighth International Conference on Advanced Computational Intelligence (ICACI), IEEE, 2016, pp. 180–185.
- [29] W. Liu, H. Zhang, Z. Ding, Q. Liu, C. Zhu, A comprehensive active learning method for multiclass imbalanced data streams with concept drift, *Knowl.-Based Syst.* 215 (2021) 106778.
- [30] C.C. Aggarwal, S.Y. Philip, J. Han, J. Wang, A framework for clustering evolving data streams, in: *Proceedings 2003 VLDB Conference*, Elsevier, 2003, pp. 81–92.
- [31] C. Yin, S. Chen, Z. Yin, Clustering-based active learning classification towards data stream, *ACM Trans. Intell. Syst. Technol.* 14 (2023) 1–18.
- [32] G. Aguiar, A. Cano, An active learning budget-based oversampling approach for partially labeled multi-class imbalanced data streams, in: *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing*, 2023, pp. 382–389.
- [33] S. Wang, L.L. Minku, X. Yao, Resampling-based ensemble methods for online class imbalance learning, *IEEE Trans. Knowl. Data Eng.* 27 (2014) 1356–1368.
- [34] A. Bernardo, E. Della Valle, Vfc-smote: very fast continuous synthetic minority oversampling for evolving data streams, *Data Min. Knowl. Discov.* 35 (2021) 2679–2713.
- [35] A. Cano, B. Krawczyk, Rose: robust online self-adjusting ensemble for continual learning on imbalanced drifting data streams, *Mach. Learn.* 111 (2022) 2561–2599.
- [36] A. Cano, B. Krawczyk, Kappa updated ensemble for drifting data stream mining, *Mach. Learn.* 109 (2020) 175–218.
- [37] A. Bifet, G. Holmes, B. Pfahringer, Leveraging bagging for evolving data streams, in: *Machine Learning and Knowledge Discovery in Databases: European Conference, Proceedings, Part I, ECML PKDD 2010, Barcelona, Spain, September 20–24, 2010*, vol. 21, Springer, 2010, pp. 135–150.
- [38] H.M. Gomes, A. Bifet, J. Read, J.P. Barddal, F. Enembreck, B. Pfahringer, G. Holmes, T. Abdesslem, Adaptive random forests for evolving data stream classification, *Mach. Learn.* 106 (2017) 1469–1495.
- [39] L.E.B. Ferreira, H.M. Gomes, A. Bifet, L.S. Oliveira, Adaptive random forests with resampling for imbalanced data streams, in: 2019 International Joint Conference on Neural Networks (IJCNN), IEEE, 2019, pp. 1–6.
- [40] W. Liu, H. Zhang, Z. Ding, Q. Liu, C. Zhu, A comprehensive active learning method for multiclass imbalanced data streams with concept drift, *Knowl.-Based Syst.* 215 (2021) 106778.